

Relatório de Trabalho Final

Cauã Galdino Garcia¹

¹ Curso de Ciência da Computação
Fundação Universidade Federal de Rondônia – UNIR
Porto Velho – RO – Brasil

{cauaagaldino}@gmail.com

Resumo. *Este trabalho foi desenvolvido para a disciplina de Estrutura de Dados II, ministrada pelo professor Thiago Guarnieri, com o objetivo de implementar uma biblioteca para manipulação de grafos em linguagem C.*

1. Introdução

Este relatório apresenta o desenvolvimento da biblioteca de grafos, descrevendo sua arquitetura, estruturas de dados utilizadas, algoritmos implementados e decisões de projeto. Também aborda as representações por matriz e lista de adjacência, grafos ponderados, estudos de caso realizados, resultados obtidos, análises de desempenho e conclusões sobre o trabalho desenvolvido.

2. Local de desenvolvimento e Análise de Estudo de caso

O trabalho foi desenvolvido em um notebook Acer Aspire equipado com processador AMD Ryzen 5 5500U e 16 GB de memória RAM, utilizando o sistema operacional Windows 11.

O ambiente de desenvolvimento escolhido foi o Visual Studio Code, com a instalação do compilador GCC por meio do MSYS2. Para facilitar a compilação e execução do projeto, foi utilizada a extensão Code Runner, configurada para compilar automaticamente todos os arquivos-fonte (.c e .h) da biblioteca e executar o programa a partir do arquivo main.c.

A biblioteca foi organizada em módulos independentes por meio de arquivos de cabeçalho (.h) e arquivos de implementação (.c), seguindo o conceito de Tipos Abstratos de Dados (TAD). Essa organização tornou o código mais modular, facilitando a manutenção, reutilização e expansão das funcionalidades desenvolvidas.

A Tabela 1 apresenta os resultados obtidos nos estudos de caso desenvolvidos ao longo deste trabalho. As colunas representam as características analisadas em cada representação do grafo, enquanto as linhas correspondem aos diferentes grafos fornecidos pelo no arquivo do trabalho.

Tabela 1. Resultados do estudo de caso da Parte 1.

Grafo	Representação	Memória (MB)	Tempo Médio BFS (ms)	Tempo Médio DFS (ms)
Grafo 1	Matriz	381.55	166.01	216.80
Grafo 1	Lista	3.43	13.26	13.64
Grafo 2	Matriz	9517.30	1482.11	1907.16
Grafo 2	Lista	40.02	138.95	138.86

Tabela 2. Estatísticas dos grafos analisados.

Grafo	Representação	Vértices	Arestas	Grau Mín.	Grau Máx.	Grau Médio
1	Matriz	10000	109923	8	43	21.98
1	Lista	10000	109923	8	43	21.98
2	Matriz	49948	1298742	1	112	52.00
2	Lista	49948	1298742	1	112	52.00

Tabela 3. Distâncias entre pares de vértices obtidas pela BFS.

Grafo	Representação	(10,20)	(10,30)	(20,30)
1	Matriz	3	3	4
1	Lista	3	3	4
2	Matriz	-1	-1	3
2	Lista	-1	-1	3

Os resultados obtidos demonstram que a representação por lista de adjacência apresentou desempenho significativamente superior em relação à matriz de adjacência, tanto no consumo de memória quanto no tempo de execução dos algoritmos BFS e DFS, principalmente para grafos esparsos.

Tabela 4. Comparação entre as implementações do algoritmo de Dijkstra.

Grafo	Implementação	Tempo Médio (100 execuções)	Melhor Desempenho
1	Vetor	519.91 ms	
	Heap	216.14 ms	Heap
2	Vetor	3753.28 ms	
	Heap	1335.64 ms	Heap

Tabela 5. Distâncias mínimas calculadas pelo algoritmo de Dijkstra a partir do vértice de origem 10.

Grafo	Origem	Destino 20	Destino 30	Destino 40	Destino 50	Destino 60
1	10	2.38	1.72	2.05	2.08	1.66
2	10	1.81	1.72	1.92	1.57	1.58

Na Parte 2, observou-se que a implementação do algoritmo de Dijkstra utilizando Heap obteve tempos de execução consideravelmente menores quando comparada à implementação baseada em vetor, confirmando a eficiência da utilização de filas de prioridade para o problema de caminhos mínimos.

3. Decisões de Projeto e Melhorias Futuras

Durante o desenvolvimento da biblioteca foram tomadas diversas decisões relacionadas à arquitetura do projeto, escolha dos algoritmos e otimização de desempenho. O principal objetivo foi desenvolver uma biblioteca organizada, modular e de fácil manutenção, permitindo que novas funcionalidades fossem adicionadas sem alterar significativamente o restante do código.

A organização foi realizada por meio da separação entre arquivos de cabeçalho (.h) e arquivos de implementação (.c), seguindo o conceito de Tipos Abstratos de Dados (TAD). Dessa forma, cada módulo ficou responsável por uma parte específica do sistema, como a manipulação de grafos não ponderados, grafos ponderados e algoritmos auxiliares. Essa divisão facilitou a realização de testes individuais, a localização de erros durante o desenvolvimento e tornou o código mais legível para futuras manutenções.

Outra decisão importante foi a utilização do algoritmo *Merge Sort* para realizar a ordenação dos graus dos vértices durante o cálculo da mediana. Como a mediana depende dos elementos estarem previamente ordenados, tornou-se necessário utilizar um algoritmo de ordenação eficiente. O *Merge Sort* foi escolhido por possuir complexidade de tempo de $O(n \log n)$ em todos os casos, apresentar comportamento estável e manter um desempenho previsível independentemente da distribuição dos dados. Embora algoritmos como *Quick Sort* também apresentem desempenho médio de $O(n \log n)$, eles podem atingir $O(n^2)$ no pior caso quando não há estratégias adicionais de escolha do pivô. Como o cálculo da mediana não é realizado repetidas vezes durante a execução do programa, optou-se por privilegiar a previsibilidade do algoritmo em vez de pequenas diferenças de desempenho.

Durante a execução dos estudos de caso também foi possível identificar oportunidades de otimização. Conforme apresentado na Tabela 1, a representação por matriz de adjacência consumiu aproximadamente 9,5 GB de memória para o segundo grafo analisado. Esse elevado consumo ocorreu porque a matriz foi implementada utilizando o tipo `int`, cujo tamanho normalmente corresponde a 4 bytes por posição. Entretanto, como a matriz armazenava apenas os valores 0 e 1 para indicar ausência ou presença de arestas, seria mais eficiente utilizar tipos como `bool` ou `char`, reduzindo significativamente a memória utilizada. Essa alteração diminuiria não apenas o espaço de armazenamento necessário, mas também poderia melhorar o desempenho devido ao menor volume de dados transferidos entre a memória principal e o processador.

A Figura 1 apresenta um momento da execução do estudo de caso utilizando o arquivo `grafo_W2.txt`, pertencente à segunda parte do trabalho. Nesse cenário é possível observar o elevado consumo de memória durante a execução do programa, consequência direta da utilização da matriz de adjacência implementada com elementos do tipo `int`. Os resultados obtidos reforçam a importância da escolha adequada das estruturas de dados e dos tipos utilizados durante a implementação, especialmente quando se trabalha com grafos de grande porte.

Name	Status	CPU	Memory	Disk	Network
main.exe		8.1%	4,776.4 MB	0 MB/s	0 Mbps
Google Chrome (19)		0.1%	946.2 MB	0.1 MB/s	0 Mbps
Visual Studio Code (11)		0.1%	610.2 MB	0 MB/s	0 Mbps
Antimalware Service Executable		0.8%	110.2 MB	0.1 MB/s	0 Mbps
WebView2 Manager (6)		0%	100.7 MB	0 MB/s	0 Mbps
Task Manager		0.8%	71.4 MB	0.1 MB/s	0 Mbps
Secure System		0%	64.1 MB	0 MB/s	0 Mbps
Windows Explorer		0%	52.1 MB	0.1 MB/s	0 Mbps
Search (7)		0%	30.2 MB	0.1 MB/s	0 Mbps
wallpaper64.exe		0%	28.8 MB	0 MB/s	0 Mbps
WebView2 Manager (6)		0%	24.5 MB	0 MB/s	0 Mbps
Desktop Window Manager		0.8%	22.2 MB	0 MB/s	0 Mbps
Service Host: Windows Health...		0.1%	21.4 MB	0 MB/s	0 Mbps
Service Host: DCOM Server Pr...		0%	17.8 MB	0.1 MB/s	0 Mbps
Service Host: State Repository ...		0%	15.9 MB	0.1 MB/s	0 Mbps
Windows Shell Experience Hos...		0%	15.7 MB	0.1 MB/s	0 Mbps
SnippingTool.exe		0%	14.8 MB	0 MB/s	0 Mbps
Service Host: Diagnostic Polic...		0%	14.8 MB	0 MB/s	0 Mbps
Start		0%	12.3 MB	0.1 MB/s	0 Mbps
Service Host: Remote Procedu...		0%	12.0 MB	0.1 MB/s	0 Mbps

Figura 1. Execução do estudo de caso utilizando o arquivo grafo_W.2.txt.

3.1. Cálculo do Consumo de Memória

A Listagem 1 apresenta a função utilizada para estimar o consumo de memória da matriz de adjacência. Foi utilizado o tipo `size_t` por ser o tipo padrão da linguagem C para representar tamanhos de objetos em memória, sendo também o tipo retornado pelo operador `sizeof`. Ao final, o valor calculado em bytes é convertido para megabytes para facilitar a análise dos resultados.

Listing 1. Função para cálculo da memória utilizada pela matriz de adjacência.

```
double calcularMemoria(Grafo *g) {
    if (g == NULL) return 0.0;

    size_t memoria = 0;

    memoria += sizeof(Grafo);
    memoria += g->numVert * sizeof(int *);

    for (int i = 0; i < g->numVert; i++) {
        memoria += g->numVert * sizeof(int);
    }

    return memoria / (1024.0 * 1024.0);
}
```

4. Conclusão

O desenvolvimento deste trabalho proporcionou uma experiência prática que complementou os conhecimentos adquiridos durante a disciplina de Estrutura de Dados II. Ao longo

da implementação foi possível compreender que decisões aparentemente simples, como a escolha do tipo de uma variável, da estrutura de dados, da representação do grafo ou do algoritmo de ordenação, podem impactar significativamente o consumo de memória, o tempo de execução e a qualidade da solução. Os resultados obtidos demonstraram, na prática, a importância de selecionar as técnicas mais adequadas para cada problema, reforçando conceitos fundamentais que certamente contribuirão para minha formação acadêmica e futura atuação profissional.

Referências